

Święta Księga C++

Tom I: Architektura Klasy Bazowej (Postac.h)

Ten dokument stanowi fundamentalny "modlitewnik" logiki w projekcie Boss Raid. Zrozumienie opisanych tu mechanizmów jest kluczem do swobodnego operowania resztą kodu i wpinania go w interfejs Qt.

1. Ochrona przed podwójnym wpięciem (Include Guards)

```
#ifndef POSTAC_H
#define POSTAC_H
// ... cały kod ...
#endif // POSTAC_H
```

To jest tzw. strażnik nagłówka. Działa jak sprzętowy bezpiecznik chroniący przed zwarcieniem w kodzie. Zapobiega sytuacji, w której kompilator próbuje wczytać tę samą definicję klasy dwa razy (np. gdy z klasy **Postac** korzysta jednocześnie klasa **Boss** i **Gracz**). Chroni to projekt przed błędem redefinicji podczas kompilacji.

2. Sekcja "Protected" (Ukryte dziedzictwo)

```
protected:
    std::string nazwa;
    int hp;
    int maxHp;
    int pancerz;
```

Standardowo zmienne ukrywa się pod słowem kluczowym **private**. Używamy tu jednak **protected**. Oznacza to: "Te zmienne są ukryte dla reszty programu, ale klasy, które z niej dziedziczą (**Boss** i **Gracz**), mają do nich pełny, bezpośredni dostęp".

3. Konstruktor (Inicjalizacja Obiektu)

```
Postac(std::string n, int startHp) {  
    nazwa = n;  
    hp = startHp;  
    maxHp = startHp;  
    pancerz = 0;  
}
```

Konstruktor odpala się automatycznie **tylko raz**, w momencie tworzenia nowego obiektu w pamięci. Przyjmuje parametry początkowe (nazwa, startowe HP) i dokonuje "ustawień fabrycznych". Gwarantuje, że żadna postać nie wejdzie do gry bez poprawnie określonego życia czy z losowymi śmieciami w zmiennej pancerza.

4. Destruktor Wirtualny (Garbage Collection)

```
virtual ~Postac() = default;
```

Jest to ukłon w stronę dobrych praktyk C++. Kiedy obiekt (np. instancja Bossa) jest niszczone i zwalnia pamięć komputera, słowo kluczowe **virtual** gwarantuje, że proces sprzątanía przebiegnie w prawidłowej kolejności — najpierw usunięte zostaną zasoby Bossa, a dopiero potem zasoby jego klasy bazowej.

5. Główna logika – Otrzymywanie obrażeń

```
void otrzymajObrazenia(int dmg) {  
    if (pancerz > 0) {  
        if (pancerz >= dmg) {  
            pancerz -= dmg;  
            dmg = 0;  
        } else {  
            dmg -= pancerz;  
            pancerz = 0;  
        }  
    }  
    hp -= dmg;  
}
```

```
    if (hp < 0) hp = 0;
}
```

Sercem mechaniki walki jest ta funkcja. Kiedy wywołana zostaje akcja uderzenia (**dmg**), algorytm przeprowadza następujące kroki:

- **Filtrowanie pancerza:** Jeśli wartość pancerza jest większa lub równa obrażeniom, pancerz absorbuje cały cios, a **dmg** spada do zera.
- **Przebicie pancerza:** Jeśli uderzenie jest potężniejsze, niszczy cały pancerz (sprowadza do zera), ale jego siła jest redukowana o zniszczoną wartość.
- **Obrażenia właściwe:** Reszta siły ciosu odejmowana jest od punktów życia (**hp**).
- **Ogranicznik dolny:** Zabezpieczenie **if (hp < 0) hp = 0;** gwarantuje, że pasek życia w Qt nigdy nie wskaże wartości ujemnej, co zapobiega błędom w interfejsie graficznym.

6. Gettery (Bezpieczny odczyt parametrów)

```
int getHp() const { return hp; }
int getMaxHp() const { return maxHp; }
int getPancerz() const { return pancerz; }
std::string getNazwa() const { return nazwa; }
```

Skoro zmienne klasowe są ukryte, interfejs Qt (paski życia, etykiety tekstowe) potrzebuje bezpiecznego sposobu na ich odczytanie. Służą do tego *Gettery*. Słowo kluczowe **const** to gwarancja dla kompilatora, która mówi: **"Ta funkcja służy wyłącznie jako czujnik odczytujący wartość i w żaden sposób nie modyfikuje stanu obiektu"**.